

```

1  import os
2  import glob
3  import time
4  import logging
5  import numpy as np
6  import torch
7  import torch.nn as nn
8  import torch.optim as optim
9  from torch.utils.data import DataLoader, random_split
10 import numpy as np
11 import matplotlib.pyplot as plt
12 import mlflow
13 import mlflow.pytorch
14 from sklearn.metrics import (
15     confusion_matrix,
16     classification_report,
17     precision_score,
18     recall_score,
19     f1_score,
20     roc_auc_score
21 )
22 import psutil
23 from mlflow.models.signature import infer_signature
24 import itertools
25 from sklearn.preprocessing import label_binarize
26
27 # Import your custom classes
28 from config_reader import ConfigReader
29 from DS_WifiCSIDataset import WifiCSIDataset
30 from DL_CSILSTMNet import CSILSTMNet
31 from DL_DenseNet1D import DenseNet1D
32 from DL_EfficientNet1DLSTM import EfficientNet1DLSTM
33 from DL_MobileNetV3 import MobileNetV3_1D_LSTM
34 # EfficientNet1D would be imported similarly
35
36 # --- Logging setup (use your CSI_ID.py pattern) ---
37 cr = ConfigReader("csi_id_config.properties")
38 now = time.strftime("%Y%m%d_%H%M%S")
39 base_dir = cr.get("local_data_path")
40 gait_filename = cr.get("file_name_for_gait")
41 filelist = glob.glob(os.path.join(base_dir, '**', gait_filename), recursive=True)
42 plot_path = f"{cr.get('output_path')}/{now}/plots"; os.makedirs(plot_path, exist_ok=True)
43 print(f"Plot path: {plot_path}")
44 log_path = f"{cr.get('output_path')}/{now}/logs"; os.makedirs(log_path, exist_ok=True)
45 print(f"Log path: {log_path}")
46 checkpoint_dir = f"{cr.get('output_path')}/{now}/checkpoints"; os.makedirs(checkpoint_dir, exist_ok=True)
47 print(f"Checkpoint path: {checkpoint_dir}")
48 log_filename = f"{log_path}/{cr.get('experiment_name')}_run_{now}.log"
49 print(f"Log file: {log_filename}")
50
51
52
53 def setup_logging(log_file_path):
54
55     for handler in logging.root.handlers[:]:
56         logging.root.removeHandler(handler)
57
58     logging.basicConfig(
59         level=logging.INFO,
60         format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
61         handlers=[
62             logging.StreamHandler(),
63             logging.FileHandler(log_file_path, mode="w"),
64         ],
65     )
66     logger = logging.getLogger()
67     logger.info("Logger initialized")
68     return logger
69
70 def train_and_evaluate(model, model_name, train_loader, val_loader, device, params,

```

```

checkpoint_dir, logger):
71     logger.info(f"START T-N-E {model_name}")
72     """Train and evaluate for one set of params, return metrics, best ckpt, and full
       history."""
73     criterion = nn.CrossEntropyLoss()
74     optimizer = optim.Adam(model.parameters(), lr=params["lr"])
75     num_epochs = params["epochs"]
76     best_val_acc, best_epoch = 0, 0
77     best_model_state = None
78     train_losses, val_losses, train_accs, val_accs = [], [], [], []
79     total_params = sum(p.numel() for p in model.parameters())
80
81     mlflow.log_param("parameter_count", total_params)
82     logger.info(f"{model_name} USING LEARNING RATE {params['lr']}")
83     for epoch in range(num_epochs):
84         start_time = time.time()
85         # Training
86         model.train()
87         running_loss, correct, total = 0.0, 0, 0
88         train_true, train_pred = [], []
89         for batch in train_loader:
90             csi_seq = batch["csi_seq"].to(device, non_blocking=True)
91             meta_seq = batch["metadata_seq"].to(device, non_blocking=True)
92             labels = batch["label"].squeeze().to(device)
93             optimizer.zero_grad()
94             outputs = model(csi_seq, meta_seq)
95             loss = criterion(outputs, labels)
96             loss.backward()
97             optimizer.step()
98             running_loss += loss.item()
99             preds = torch.argmax(outputs, dim=1)
100            correct += (preds == labels).sum().item()
101            total += labels.size(0)
102
103            train_true.extend(labels.cpu().numpy())
104            train_pred.extend(preds.cpu().numpy())
105
106            train_loss = running_loss / len(train_loader)
107            train_acc = correct / total
108
109            train_precision = precision_score(train_true, train_pred, average="weighted",
110            zero_division=0)
111            train_recall = recall_score(train_true, train_pred, average="weighted",
112            zero_division=0)
113            train_f1 = f1_score(train_true, train_pred, average="weighted", zero_division=
114            0)
115
116            tra_metrics_to_log = {
117                "Training Loss Curve": train_loss,
118                "Training Accuracy Curve": train_acc,
119                "Training Precision Curve": train_precision,
120                "Training Recall Curve": train_recall,
121                "Training F1 Score": train_f1
122            }
123            mlflow.log_metrics(tra_metrics_to_log, step=epoch)
124
125            # Validation phase
126            model.eval()
127            running_loss, correct, total = 0.0, 0, 0
128            val_true, val_pred, val_prob = [], [], []
129
130            with torch.no_grad():
131                for batch in val_loader:
132                    csi_seq = batch["csi_seq"].to(device, non_blocking=True)
133                    meta_seq = batch["metadata_seq"].to(device, non_blocking=True)
134                    labels = batch["label"].squeeze().to(device, non_blocking=True)
135                    outputs = model(csi_seq, meta_seq)
136                    loss = criterion(outputs, labels)
137                    running_loss += loss.item()
138                    preds = torch.argmax(outputs, dim=1)
139                    correct += (preds == labels).sum().item()

```

```

138         total += labels.size(0)
139
140         val_true.extend(labels.cpu().numpy())
141         val_pred.extend(preds.cpu().numpy())
142         val_prob.extend(torch.softmax(outputs, dim=1).cpu().numpy())
143
144     val_loss = running_loss / len(val_loader)
145     val_acc = correct / total
146
147     val_precision = precision_score(val_true, val_pred, average="weighted",
148                                     zero_division=0)
149     val_recall = recall_score(val_true, val_pred, average="weighted",
150                               zero_division=0)
151     val_f1 = f1_score(val_true, val_pred, average="weighted", zero_division=0)
152
153     val_metrics_to_log = {
154         "Validation Loss Curve": val_loss,
155         "Validation Accuracy Curve": val_acc,
156         "Validation Precision Curve": val_precision,
157         "Validation Recall Curve": val_recall,
158         "Validation F1 Score": val_f1
159     }
160     mlflow.log_metrics(val_metrics_to_log, step=epoch)
161
162     val_true_np = np.array(val_true)
163     val_prob_np = np.array(val_prob)
164
165     unique_classes = np.unique(val_true_np)
166     num_classes = val_prob_np.shape[1]
167
168     if len(unique_classes) == num_classes:
169         # All classes present, compute directly
170         try:
171             val_auc = roc_auc_score(val_true_np, val_prob_np, multi_class='ovr',
172                                     average='weighted')
173             mlflow.log_metric('Validation AUC Curve', val_auc, step=epoch)
174         except Exception as e:
175             logger.warning(f"ROC-AUC computation failed: {e}")
176     else:
177         # Subset classes and probabilities to avoid mismatch error
178         try:
179             y_true_bin = label_binarize(val_true_np, classes=unique_classes)
180             val_prob_subset = val_prob_np[:, unique_classes]
181             val_auc = roc_auc_score(y_true_bin, val_prob_subset, multi_class='ovr',
182                                     average='weighted')
183             mlflow.log_metric('Validation AUC Curve', val_auc, step=epoch)
184         except Exception as e:
185             logger.warning(f"ROC-AUC subset computation failed: {e}")
186
187     for param_group in optimizer.param_groups:
188         mlflow.log_metric("Learning Rate Over Epochs", param_group["lr"], step=
189                             epoch)
190
191     end_time = time.time()
192
193     hw_one_metrics = {
194         "Epoch Duration in Seconds": end_time - start_time,
195         "CPU Utilization Percent": psutil.cpu_percent(),
196         "Memory Used GB": psutil.virtual_memory().used / (1024 ** 3),
197         "Total Memory GB": psutil.virtual_memory().total / (1024 ** 3)
198     }
199     mlflow.log_metrics(hw_one_metrics, step=epoch)
200
201     disk_info = psutil.disk_usage('/')
202     disk_used = disk_info.used / (1024 ** 3) # Convert bytes to GB
203     disk_total = disk_info.total / (1024 ** 3) # Convert bytes to GB
204     logger.debug(f"Disk usage: {disk_used:.2f} GB used out of {disk_total:.2f} GB
205                  total")
206
207     hw_two_metrics = {
208         "Disk Usage GB": disk_used,
209         "Total Disk Space GB": disk_total

```

```

204     }
205     mlflow.log_metrics(hw_two_metrics, step=epoch)
206
207     logger.info(
208         f"Epoch {epoch+1}/{num_epochs}, "
209         f"Model {model_name}, "
210         f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.4f}, Val Loss: {
211         val_loss:.4f}, Val Acc: {val_acc:.4f}, "
212         f"Epoch Duration: {end_time-start_time:.2f} sec"
213     )
214
215     # Save best model
216     if val_acc > best_val_acc:
217         best_val_acc = val_acc
218         best_epoch = epoch + 1
219         best_model_state = model.state_dict()
220
221     # --- Log the final model with signature ---
222     # signature = infer_signature()
223
224     mlflow.pytorch.log_model(
225         pytorch_model=model,
226         artifact_path=f"best_model_{model_name}.{params['model_name']}",
227         # signature=signature
228     )
229
230     train_losses.append(train_loss)
231     val_losses.append(val_loss)
232     train_accs.append(train_acc)
233     val_accs.append(val_acc)
234
235     learning_rate = params['lr']
236     logger.fatal(f"Training complete. Best Val Acc: {best_val_acc:.4f} at epoch {
237     best_epoch}.")
238
239     if best_model_state is not None:
240         best_model_fname = f"{checkpoint_dir}/best_model_{params['model_name']}_epoch{
241         best_epoch}.pt"
242         torch.save(best_model_state, best_model_fname)
243         logger.fatal(f"SAVED BEST MODEL {best_model_fname} USING LEARNING RATE {
244         learning_rate}")
245         logger.fatal(f"BEST VALIDATION ACCURACY: {best_val_acc:.4f} at epoch {
246         best_epoch}. ")
247
248     mlflow.log_artifact(best_model_fname)
249
250     return train_losses, val_losses, train_accs, val_accs, best_model_state,
251     best_val_acc, best_epoch, learning_rate
252
253
254 def plot_stats(history, save_path, logger):
255     epochs = range(1, len(history['accuracy']) + 1)
256     plt.figure(figsize=(12, 5))
257     plt.subplot(1, 2, 1)
258     plt.plot(epochs, history['accuracy'], label='Train Accuracy')
259     plt.plot(epochs, history['val_accuracy'], label='Validation Accuracy')
260     plt.title('Accuracy over Epochs')
261     plt.legend(); plt.grid(True)
262     plt.subplot(1, 2, 2)
263     plt.plot(epochs, history['loss'], label='Train Loss')
264     plt.plot(epochs, history['val_loss'], label='Validation Loss')
265     plt.title('Loss over Epochs')
266     plt.legend(); plt.grid(True)
267     plt.tight_layout()
268     plt.savefig(save_path)
269     mlflow.log_artifact(save_path)
270     logger.info(f"Saved stats plot to {save_path}")
271     plt.close()
272
273 import seaborn as sns
274
275 import matplotlib.pyplot as plt
276 import seaborn as sns

```

```

270 import numpy as np
271
272 import matplotlib.pyplot as plt
273 import seaborn as sns
274 import numpy as np
275
276 def plot_cm_cr(cm, cr_report, model_name, params, plot_path):
277     # --- Robust Confusion Matrix Plot ---
278     fig_width = max(12, 0.5 * cm.shape[0])
279     fig_height = max(9, 0.5 * cm.shape[0])
280     plt.figure(figsize=(fig_width, fig_height))
281     sns.heatmap(
282         cm,
283         annot=True,
284         fmt='d',
285         cmap='Blues',
286         cbar=True,
287         annot_kws={"size": 6 if cm.shape[0] > 20 else 10}
288     )
289     plt.title('Confusion Matrix', fontsize=16)
290     plt.ylabel('True Label', fontsize=14)
291     plt.xlabel('Predicted Label', fontsize=14)
292     tick_font_size = 6 if cm.shape[0] > 20 else 9
293     plt.xticks(np.arange(cm.shape[1]) + 0.5, np.arange(1, cm.shape[1]+1), rotation=90,
294               fontsize=tick_font_size)
295     plt.yticks(np.arange(cm.shape[0]) + 0.5, np.arange(1, cm.shape[0]+1), rotation=0,
296               fontsize=tick_font_size)
297     plt.tight_layout()
298     cm_path = f"{plot_path}/{model_name}_{str(params)}_confusion_matrix.png"
299     plt.savefig(cm_path, bbox_inches='tight', dpi=150)
300     plt.close()
301
302     # --- Robust Classification Report Plot ---
303     # Always treat as a string for wide compatibility, autoscale height for #lines
304     report_str = str(cr_report)
305     n_lines = report_str.count('\n') + 1
306     plt.figure(figsize=(12, min(max(n_lines * 0.4, 6), 48)))
307     plt.text(0, 1, report_str, fontsize=10, family='monospace', verticalalignment=
308            'top')
309     plt.axis('off')
310     plt.title('Classification Report')
311     cr_path = f"{plot_path}/{model_name}_{str(params)}_classification_report.png"
312     plt.savefig(cr_path, bbox_inches='tight')
313     plt.close()
314
315     # Log artifacts to MLflow
316     import mlflow
317     mlflow.log_artifact(cm_path)
318     mlflow.log_artifact(cr_path)
319
320 def run_mlop_pipeline():
321     logger = setup_logging(log_filename)
322     print(f"logger {logger}")
323     # --- Dataset loading (as in DS_WifiCSIDataset.py) ---
324     dataset = WifiCSIDataset(logger, filelist, window_size=128, stride=64)
325     logger.info(f"Dataset loaded with {len(dataset)} samples")
326
327     train_size = int(0.8 * len(dataset))
328     test_size = len(dataset) - train_size
329     train_dataset, test_dataset = random_split(dataset, [train_size, test_size])
330
331     train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
332     test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)
333
334     device = torch.device("mps" if torch.backends.mps.is_available() else
335                           "cuda" if torch.cuda.is_available() else "cpu")
336
337     logger.info(f"Using device: {device}")
338     # --- Model Definitions & hyperparameter grid ---
339     model_defs = {
340         "CSILSTMNet": (CSILSTMNet, {'csi_input_size':[99], 'meta_input_size':[12],
341                                     'window_size':[128], 'num_classes':[31], 'lr':[0.001, 0.0005], 'epochs':[cr.

```

```

    get_int("epochs"))}},
338 "DenseNet1D": (DenseNet1D, {'csi_channels':[99], 'meta_feature_dim':[12],
    'num_classes':[31], 'lr':[0.001, 0.0005], 'epochs':[cr.get_int("epochs")])),
339 "EfficientNet1DLSTM": (EfficientNet1DLSTM, {'in_channels':[99], 'meta_seq_len'
    :[128], 'meta_feature_dim':[12], 'num_classes':[31], 'lr':[0.001, 0.0005],
    'epochs':[cr.get_int("epochs")])),
340 "MobileNetV3_1D_LSTM": (MobileNetV3_1D_LSTM, {'csi_channels':[99],
    'meta_feature_dim':[12], 'num_classes':[31], 'lr':[0.001, 0.0005], 'epochs':[
    cr.get_int("epochs")])),
341 # If EfficientNet1D is available, add here
342 }
343 learning_rate = None
344 # --- MLflow experiment ---
345 mlflow.set_experiment(cr.get("experiment_name"))
346 best_overall = {"val_acc":-1}
347 stats_summary = {}
348 best_model_path = None
349 for model_name, (model_class, param_grid) in model_defs.items():
350     param_keys, param_vals = zip(*param_grid.items())
351     logger.info(" ")
352     logger.info("N")
353     logger.info("E")
354     logger.info("W")
355     logger.info(f"STARTING TRAINING FLOW : {model_name}:{param_keys}:{param_vals}")
356     with mlflow.start_run(run_name=f"{model_name}_main") as parent_run:
357         for combo in itertools.product(*param_vals):
358             params = dict(zip(param_keys, combo)); params['model_name'] =
                model_name

359
360             logger.info(f"START RUN MLFLOW : {model_name}:{str(params)}")
361             batch = next(iter(train_loader))
362             # Model instantiation according to constructor
363             if model_name == "CSILSTMNet":
364                 model = model_class(
365                     csi_input_size=batch["csi_seq"].shape[2],
366                     meta_input_size=batch["metadata_seq"].shape[2],
367                     window_size=batch["metadata_seq"].shape[1],
368                     num_classes=31
369                 ).to(device, non_blocking=True)
370             elif model_name in ["DenseNet1D", "MobileNetV3_1D_LSTM"]:
371                 model = model_class(
372                     csi_channels=batch["csi_seq"].shape[2],
373                     meta_feature_dim=batch["metadata_seq"].shape[2],
374                     num_classes=31
375                 ).to(device, non_blocking=True)
376             elif model_name == "EfficientNet1DLSTM":
377                 model = model_class(
378                     csi_input_channels=batch["csi_seq"].shape[2],
379                     meta_input_size=batch["metadata_seq"].shape[-1],
380                     num_classes=31
381                 ).to(device, non_blocking=True)
382             else:
383                 raise ValueError("Unknown model")
384             with mlflow.start_run(run_name=f"{model_name}_lr_{params['lr']}",
                nested=True) as child_run:
385                 # mlflow.log_params(params)
386                 train_losses, val_losses, train_accs, val_accs, best_model_state,
                best_val_acc, best_epoch, learning_rate = train_and_evaluate(
                model, model_name, train_loader, test_loader, device, params,
                checkpoint_dir, logger)
387             #logger.info(f"best model {best_model_state}")
388             history = {
389                 'accuracy': train_accs,
390                 'val_accuracy': val_accs,
391                 'loss': train_losses,
392                 'val_loss': val_losses
393             }
394         stats_summary[(model_name, str(params))] = {
395             "val_acc": best_val_acc,
396             "epoch": best_epoch,
397             "history": history
398         }

```

```

399     }
400     plot_stats(history, save_path=f"{plot_path}/{model_name}_{str(params)}_stats.png", logger=logger)
401     # Save confusion matrix, classification report on test set
402     if best_model_state is None:
403         logger.warning("No best model state found, skipping test evaluation.")
404         continue
405     model.load_state_dict(best_model_state)
406     all_preds, all_labels = [], []
407     model.eval()
408     with torch.no_grad():
409         for batch in test_loader:
410             csi_seq = batch["csi_seq"].to(device, non_blocking=True)
411             meta_seq = batch["metadata_seq"].to(device, non_blocking=True)
412             labels = batch["label"].squeeze().cpu().numpy()
413             outputs = model(csi_seq, meta_seq)
414             preds = torch.argmax(outputs, dim=1).cpu().numpy()
415             all_preds.extend(preds)
416             all_labels.extend(labels)
417     cm = confusion_matrix(all_labels, all_preds)
418     cr_report = classification_report(all_labels, all_preds)
419     np.save(f"{plot_path}/{model_name}_{str(params)}_cm.npy", cm)
420     mlflow.log_artifact(f"{plot_path}/{model_name}_{str(params)}_cm.npy")
421     plot_cm_cr(cm, cr_report, model_name, params, plot_path)
422
423     logger.info(f"Confusion matrix:\n{cm}")
424     logger.info(f"Classification report:\n{cr_report}")
425     # Save model checkpoint for best overall if needed
426     if best_val_acc > best_overall["val_acc"]:
427         best_overall = {"model": model_name, "params": params, "val_acc":
428             best_val_acc, "epoch": best_epoch, "lr": learning_rate}
429         torch.save(best_model_state, f"{checkpoint_dir}/best_model_{params['model_name']}_epoch{best_epoch}.pt")
430         logger.fatal(f"99. SAVED BEST MODEL IN {model_name}. BEST VALIDATION ACCURACY: {best_val_acc:.4f} at epoch {best_epoch}.")
431         best_model_path = f"{checkpoint_dir}/best_model_{params['model_name']}_epoch{best_epoch}.pt"
432         mlflow.log_artifact(f"{best_model_path}")
433
434         logger.fatal(f"100. LOGGED BEST MODEL IN MLFLOW {best_model_path}. BEST VALIDATION ACCURACY: {best_val_acc:.4f} at epoch {best_epoch}.")
435
436     mlflow.log_metric("Top Validation Accuracy", best_val_acc)
437     # print(f"Best model overall: {best_overall}")
438     '''
439     logger.info(f"Best model overall: {best_overall}")
440     mlflow.log_params(stats_summary)
441     logger.info(f"Logged stats summary to MLflow {stats_summary}")
442
443     mlflow.log_params(best_overall)
444     logger.info(f"Logged best overall to MLflow {best_overall}")
445     '''
446     mlflow.log_artifact(best_model_path)
447     logger.info(f"Logged best model path to MLflow {best_model_path}")
448     logger.fatal(f"best_overall : {best_overall}")
449 if __name__ == "__main__":
450     run_mlop_pipeline()
451
452 # for additional metrics -
453 #

```